

目录

基础.....	1
关系模型.....	2
数据完整性约束.....	2
关系运算 (Relational algebra)	2
ER model (conceptual level)	4
Data model	5
EER?.....	5
Logical Database Design(ER->Relational)	5
Database design	6
Sql.....	8
Normalization	11
Transaction	15
Distributed DBMSs	18
数据库安全+数据伦理.....	23
可扩展标记语言 XML.....	23
NoSQL.....	28

基础

什么是 data
什么是 database
什么是 DBMS
什么是 data model
什么是 data schema
DBMS 的构成

Different roles in Database Environment

1. Data Administrator
2. Database Administrator
3. Database Designer
4. Application Developer
5. End User

ANSI-SPARC architecture

External

Conceptual

Internal

Physical data organization+画图

为什么要设计 ANSI-SPARC architecture

什么是 system catalogue/meta data/ Data Dictionary (都一个意思)

Logically related data 包含什么(感觉就是 er model 需要的东西)

什么是 entity, attributes, relationship.

使用 DBS 的优点

关系模型

Relation

Attribute

Tuple

Degree

Cardinality

Domain (the set of allowable values for one or more attributes)

Relational model(就是数据表格的集合 (很多个 table))

Relational database(经过 normalized 之后的)

Relational schema 【A relation can be represented in this format below: Relation name(attribute 1, attribute 2, ... attribute n)】

Relational algebra (formal language)

什么是 null

什么是 candidate key

什么是 primary key

什么是 foreign key (注意 match 的对象类型)

什么是 composite key (candidate key that consists of two or more attributes) (应该不考)

Candidate key 的两条性质

Uniqueness

Irreducibility

数据完整性约束

Entity integrity (主键不为空)

Referential integrity (外键要么 match 要么为空)

关系运算 (Relational algebra)

Basic operation

·Select(横着选): List all staff with a salary greater than £10,000.

$\sigma_{\text{salary} > 10000 \wedge \text{sex} = \text{"F"}}(\text{Staff})$ $\sigma_{\text{city} = \text{"London"}}(\text{Branch})$ $\sigma_{\text{salary} > 10000}(\text{Staff})$

·Projection (竖着选)

$\Pi_{\text{staffNo}, \text{fName}, \text{lName}, \text{salary}}(\text{Staff})$ Answer 1: $\sigma_{\text{salary} > 20000}(\Pi_{\text{staffNo}, \text{position}, \text{salary}}(\text{Staff}))$
 Answer 2: $\Pi_{\text{staffNo}, \text{position}, \text{salary}}(\sigma_{\text{salary} > 20000}(\text{Staff}))$

·cartesian product (完全乘起来)

Name	ID	Course	Year
Yang Wang	20184123	IoT	2018
Yixin Xi	20184113		2018
Zhou Li	20164888	Telecomm	2019

Name	ID	Course	Year
Yang Wang	20184123	IoT	2018
Yixin Xi	20184113	IoT	2018
Zhou Li	20164888	IoT	2018
Yang Wang	20184123	Telecomm	2019
Yixin Xi	20184113	Telecomm	2019
Zhou Li	20164888	Telecomm	2019

R X S

·join

Natural join

就是 join on 最后一个 attribute

$\Pi_{\text{staffNo}}(\sigma_{\text{city} = \text{"London"}}(\text{Staff} \bowtie \text{Branch}))$

T		U		T ⋈ U		
A	B	B	C	A	B	C
a	1	1	x	a	1	x
b	2	1	y	a	1	y
		3	z			

Theta join $R \bowtie_F S = \sigma_F(R \times S)$

Set operation

Union

$R \cup S$

–Union of two relations R and S defines a relation that contains all the tuples of R, or S, or both R and S, duplicate tuples being eliminated.

–R and S must be union-compatible.

• Union compatible: Same number of attributes and corresponding attributes have the same domain.

Union example

Relation: Students1		Relation: Students2	
Name	ID	Name	ID
Yang Wang	20184123	Zhou Li	20164888
Yixin Xi	20184113	Hengda Yin	20161333

Students1 $\cup$ Students2

Name	ID
Yang Wang	20184123
Yixin Xi	20184113
Zhou Li	20164888
Hengda Yin	20161333

Set difference

$R - S$

– Defines a relation consisting of the tuples that are in relation R, but not in S.

– R and S must be union-compatible.

Set Difference example

Relation: Students1		Relation: Students2	
Name	ID	Name	ID
Yang Wang	20184123	Zhou Li	20164888
Yixin Xi	20184113	Hengda Yin	20161333
Zhou Li	20164888		
Hengda Yin	20161333		

Students1 – Students2

Name	ID
Yang Wang	20184123
Yixin Xi	20184113

Intersection

- $R \cap S$
 - Defines a relation consisting of the set of all tuples that are in both R and S.
 - R and S must be union-compatible.
- Expressed using basic operations: $R \cap S = R - (R - S)$

Intersection example

Relation: Students1		Relation: Students2	
Name	ID	Name	ID
Yang Wang	20184123	Hengda Yin	20161333
Yixin Xi	20184113	Yang Wang	20184123
Zhou Li	20164888	Yixin Xi	20184113

Students1 $\cap$ Students2	
Name	ID
Yang Wang	20184123
Yixin Xi	20184113

Relational Algebra Exercise

Given the following relational schema:
 Hotel (hotelNo, hotelName, city)
 Room (roomNo, hotelNo, type, price)
 Booking (hotelNo, guestNo, dateFrom, dateTo, roomNo)
 Guest (guestNo, guestName, guestAddress)

Formulate the following queries in relational algebra:

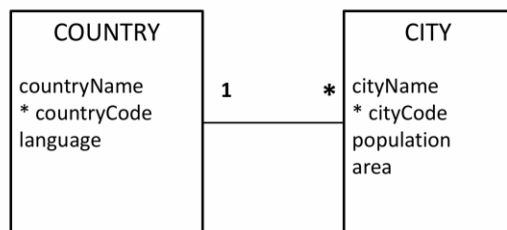
1. List all details of all rooms.
2. List all hotels in Beijing.
3. Give the hotel number of those hotels with a room price greater than £50.
4. List the price and types of all rooms in Happy Hotel.
5. Produce a relation containing all rooms at all hotels.
6. Give all hotel names with a room price above £50.
7. List the guest number of all guests currently staying at the Happy Hotel. (Hint: use current_date() for today's date)
8. List guest number of all guests who have ever booked a hotel in Beijing.
9. List guest number and name of those guests who have stayed in executive rooms.

Sample solution

1. Room
2. $\sigma_{city = "Beijing"}(Hotel)$
3. $\pi_{hotelNo}(\sigma_{price > 50}(Room))$
4. $\pi_{price, type}(\sigma_{hotelName = "Happy"}(Hotel \bowtie Room))$
5. Hotel $\bowtie$ Room
6. $\pi_{hotelName}(\sigma_{price > 50}(Hotel \bowtie Room))$
7. $R1 = \sigma_{hotelName = "Happy" \wedge dateFrom < current_Date() \wedge dateTo > current_Date()}(Hotel \bowtie Room)$
 Final result: $\pi_{guestNo}(R1)$
8. $\pi_{guestNo}(\sigma_{city = "Beijing"}(Booking \bowtie Hotel))$
9. $\pi_{guestNo, guestName}(\sigma_{type = "Executive"}(Guest \bowtie Booking \bowtie Room))$

ER model (conceptual level)

什么是 Entity-relationship modeling

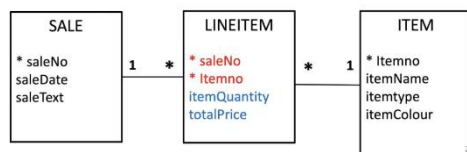


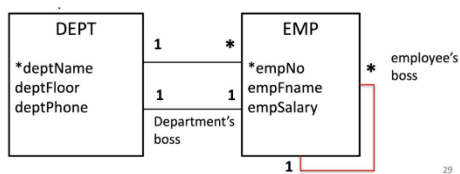
其中的三个 main concepts 是什么

什么是 recursive relationship

什么是 multiplicity

什么是 relationship degree (binary ternary)





Data model

什么是 data modelling

A technique for modelling data

A graphical representation of a database

Data model 的目标是

Identify the facts to be stored in the database

A well formed data model

A high fidelity image

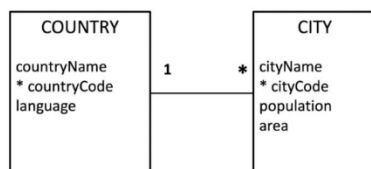
EER?

Logical Database Design(ER->Relational)

Mapping ER model concepts to relation

- Entity
- Binary 1:1, 1:m, m:m relationships
- Complex relationships
- Multi-valued attributes

1. 1:*

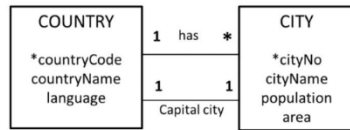


Country(countryCode, countryName, language)

City(cityCode, cityName, population, area, **countryCode**)

因为一个国家有很多 city，所以在 city 那里面写上它属于哪个国家

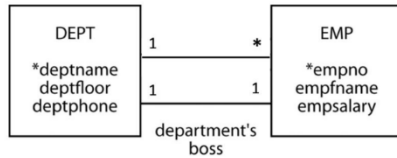
2. 1:1



COUNTRY(countryCode, countryName, language, cityNo)

CITY(cityNo, cityName, population, area, countryCode)

再来个例子:



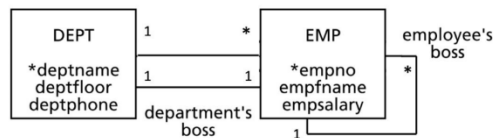
Dept(deptname, deptfloor, deptphone, empno) 写 1:1 的 key

Emp(empno, empfname, empsalary, deptname) 写 1:* 的 key

因为一个部门有好多员工, 所以给每个员工那里写上他属于哪个部门

然后又因为 boss 是 1:1 的, 所以要在部门那里写上 boss 是谁

3. 1:* 且 recursive 的

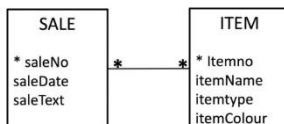


Dept(deptname, deptfloor, deptphone, empno) 写 1:1 的 key

Emp(empno, empfname, empsalary, deptname, bossempno) 写 1:* 的 key

每个员工都有一个 boss, 所以给他们都写上他们的 boss 是谁

5. *: * 的



SALE(saleNo, saleDate, saleText)

ITEM(itemno, itemName, itemtype, itemColour)

LINEITEM(saleNo, itemNo, itemQuantity, totalPrice)

最后又加上了两个自己觉得合理的

Database design

什么是 database design methodology

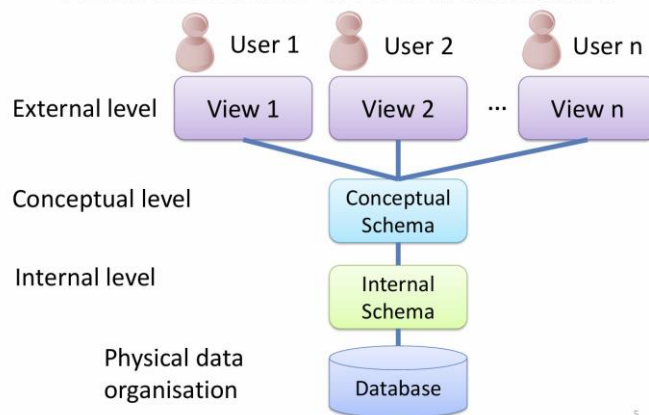
A structured approach that uses procedures, techniques, tools, and documentation aids to support and facilitate the process of design

Database design 的 四个 main phases

1. Gather requirements
 - Documents questionnaire meetings
2. Conceptual database design

3. Logical database design
4. Physical database design

Three level ANSI-SPARC architecture



Conceptual database design

Step 1 Build conceptual data model

- Step 1.1 Identify entity types
- Step 1.2 Identify relationship types
- Step 1.3 Identify and associate attributes with entity or relationship types
- Step 1.4 Determine attribute domains
- Step 1.5 Determine candidate, primary, and alternate key attributes
- Step 1.6 Consider use of enhanced modeling concepts (optional step)
- Step 1.7 Check model for redundancy
- Step 1.8 Validate conceptual model against user transactions
- Step 1.9 Review conceptual data model with user

Logical database design

Step 2 Build and validate logical data model

- Step 2.1 Derive relations for logical data model
- Step 2.2 Validate relations using normalization
- Step 2.3 Validate relations against user transactions
- Step 2.4 Check integrity constraints

To check integrity constraints are represented in the logical data model. This includes identifying:

- Required data (value not allowed to be null)
 - Attribute domain constraints
 - Multiplicity (*:* relationships)
 - Entity integrity (primary key can not be null)
 - Referential integrity (foreign keys)
 - General constraints
- Step 2.5 Review logical data model with user
 - Step 2.6 Merge logical data models into global model (optional step)
 - Step 2.7 Check for future growth

Sql

什么是 sql

A transform-oriented language with 2 major components:

- A DDL(data definition) for defining database structure.(create drop table)
- A DML(data manipulation) for retrieving and updating data(insert delete update select)

例子

SELECT staffNo, fName, lName, position, salary FROM Staff WHERE salary > 10000;

SELECT staffNo, fName, lName, position, salary FROM Staff WHERE salary BETWEEN 20000 AND 30000;

SELECT ownerNo, fName, lName, address, telNo FROM PrivateOwner WHERE address LIKE “%Glasgow%”;字符串匹配

SELECT staffNo, fName, lName, salary FROM Staff ORDER BY salary DESC/ASC;降序/升序

SELECT propertyNo, type, rooms, rent FROM PropertyForRent ORDER BY type, rent DESC;

propertyNo	type	rooms	rent
PG16	Flat	4	450
PL94	Flat	4	400
PG36	Flat	3	375
PG4	Flat	3	350
PA14	House	6	650
PG21	House	5	600

字母表默认上面为 a，数字得指明升降序

SELECT COUNT(DISTINCT propertyNo) AS myCount FROM Viewing WHERE viewDate BETWEEN “2004-05-01” AND “2004-05-31”;count 应用

SELECT COUNT(staffNo) AS myCount, SUM(salary) AS mySum FROM Staff WHERE position = “Manager”;sum 应用

SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum FROM Staff GROUP BY branchNo ORDER BY branchNo;group by

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00

SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum FROM Staff GROUP BY branchNo HAVING COUNT(staffNo) > 1 ORDER BY branchNo; **having**

List staff who work in branch at '163 Main St'

SELECT staffNo, fName, lName, position FROM Staff WHERE branchNo = (SELECT branchNo FROM Branch WHERE street = '163 Main St'); **子查询**

List all staff whose salary is greater than the average salary, and show by how much

SELECT staffNo, fName, lName, position, salary - (SELECT AVG(salary) FROM Staff) As SalDiff FROM Staff WHERE salary > (SELECT AVG(salary) FROM Staff); **又一个子查询**

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

Find number of properties handled by each staff member.

SELECT s.branchNo, s.staffNo, COUNT(*) AS myCount FROM Staff s, PropertyForRent p WHERE s.staffNo = p.staffNo GROUP BY s.branchNo, s.staffNo ORDER BY s.branchNo, s.staffNo; **多个 group by 组合**

branchNo	staffNo	myCount
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

List properties handled by staff at '163 Main St'.

SELECT propertyNo, street, city, postcode, type, rooms, rent FROM PropertyForRent WHERE staffNo IN (SELECT staffNo FROM Staff WHERE branchNo = (SELECT branchNo FROM Branch WHERE street = "163 Main St")); **in 应用**

propertyNo	street	city	postcode	type	rooms	rent
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600

Find staff whose salary is larger than salary of at least one member of staff at branch B003.

SELECT staffNo, fName, lName, position, salary FROM Staff WHERE salary > SOME (SELECT salary FROM Staff WHERE branchNo = 'B003'); 只要 some 中的一个值满足要求, 就返回真

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

Find staff whose salary is larger than salary of every member of staff at branch B003.

SELECT staffNo, fName, lName, position, salary FROM Staff WHERE salary > ALL (SELECT salary FROM Staff WHERE branchNo = 'B003'); 只有 all 中的所有值都满足条件才返回真

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00

Find all staff who work in a London branch.

SELECT staffNo, fName, lName, position FROM Staff s WHERE EXISTS (SELECT * FROM Branch b WHERE s.branchNo = b.branchNo AND city = 'London'); exist 应用

List the propertyNo of all properties that have never been viewed by any client

SELECT p.propertyNo FROM PropertyForRent p WHERE NOT EXISTS (SELECT * FROM Viewing v WHERE v.propertyNo = p.propertyNo); not exist 应用

DML - UPDATE

UPDATE 表名

SET 列 1 = 新值 1, 列 2 = 新值 2,

WHERE 条件;

Eg:

UPDATE Staff SET salary = salary*1.03;

DML - DELETE

DELETE FROM TableName [WHERE searchCondition]

DELETE FROM Viewing WHERE propertyNo = 'PG4';

DML – INSERT

INSERT INTO Staff VALUES ('SL21', 'John', 'White', 'Manager', 'M', '1945-10-01', 30000, 'B005');

- Insert a new row into Staff table supplying data for all mandatory columns.

INSERT INTO Staff (staffNo, fName, lName, position, salary, branchNo) VALUES ('SG44', 'Anne', 'Jones', 'Assistant', 8100, 'B003');

- Or

INSERT INTO Staff VALUES ('SG44', 'Anne', 'Jones', 'Assistant', NULL, NULL, 8100, 'B003');

DDL – Create Table

```
CREATE TABLE `branch` (
  `BranchNo` char(4) NOT NULL,
  `street` varchar(45) DEFAULT NULL,
  `city` varchar(45) DEFAULT NULL,
  `postcode` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`BranchNo`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8
```

DDL – Drop Table

Normalization

什么是 Normalization

Normalization 的优点与缺点

- Normalization
 - Removes data redundancy
 - Solves INSERT, UPDATE, and DELETE anomalies
 - This makes it easier to maintain the information in the database in a consistent state
- However
 - It leads to more tables in the database
 - Often these need to be joined back together, which is expensive to do
 - So sometimes (not often) it is worth '**denormalizing**'
- Relations that contain redundant information may have problems called **update anomalies**:
 - Insertion anomalies
 - what will happen if we want to insert details of new staff of B007?
 - What will happen if we want to insert details of a new branch with no staff?
 - Deletion anomalies
 - What will happen if we delete Mary Howe from StaffBranch?
 - Modification anomalies
 - What will happen if we want to change B003's address?

De~

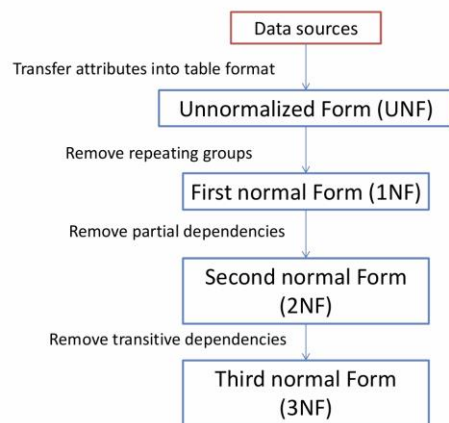
Functional dependency (Full/Partial)

Transitive dependency

Multi-valued dependency

范式的等级和过程

The Process of Normalization



UNF example

ClientRental

Client No	cName	property No	pAddress	rentStart	rentFinish	rent	owner No	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-07	31-Aug-08	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-08	1-Sep-09	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-06	10-Jun-07	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-07	1-Dec-08	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-09	10-Aug-10	450	CO93	Tony Shaw

1NF: 每行和每列的交集包含且仅包含一个值的关系

2NF: 关系必须在 1NF 中，每个非主键属性完全函数依赖于主键。

3NF: 关系必须在 2NF 中，没有非主键属性传递依赖于主键。

BCNF: 当且仅当每个决定因素都是候选键，那么这个关系在 BCNF

4NF: 一个关系在 4NF 中，当且仅当每个非平凡的多值依赖 $A \twoheadrightarrow B$ 中，A 是该关系的候选键

1NF to 2NF

Client No	property No	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
-----------	-------------	-------	----------	-----------	------------	------	---------	-------

Step 1: identify functional dependencies

fd1 clientNo, propertyNo -> rentStart, rentFinish

fd2 clientNo -> cName

fd3 propertyNo -> pAddress, rent, ownerNo, oName

fd4 ownerNo -> oName

fd5 clientNo, rentStart -> propertyNo, pAddress, rentFinish, rent, ownerNo, oName

fd6 propertyNo, rentStart -> clientNo, cName, rentFinish

Step 2: identify primary key

Step 3: identify partial dependencies on primary key, then remove them

Fd2 and fd3 are partial dependencies on primary key.

Client (clientNo, cName)

PropertyOwner(propertyNo, pAddress, rent, ownerNo, oName)

Rental(clientNo, propertyNo, rentStart, rentFinish)

2NF to 3NF

Step 1: identify functional dependencies

Step 2: identify primary key

Client (clientNo, cName)

clientNo -> cName

Rental(clientNo, propertyNo, rentStart, rentFinish)

clientNo, propertyNo -> rentStart, rentFinish (primary key)

clientNo, rentStart -> propertyNo, rentFinish (candidate key)

propertyNo, rentStart -> clientNo, rentFinish (candidate key)

PropertyOwner(propertyNo, pAddress, rent, ownerNo, oName)

propertyNo -> pAddress, rent, ownerNo, oName

ownerNo -> oName

(transitive dependency on primary key)

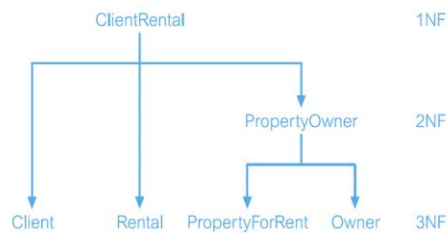
Step 3: identify transitive dependencies on primary key,
then remove them

Client (clientNo, cName)

Rental(clientNo, propertyNo, rentStart, rentFinish)

PropertyForRent(propertyNo, pAddress, rent,
ownerNo)

Owner(ownerNo, oName)



一个例子

Examine the table shown below. This table represents the hours worked per week for temporary staff at each branch of a company. Assume for a staff working at a branch, the position and hours per week is fixed.

staffNo	branchNo	branchCity	name	position	hoursPerWeek
S4555	B002	Seattle	Ellen Layman	Assistant	16
S4555	B004	Seattle	Ellen Layman	trainee	10
S4612	B002	Seattle	Dave Sinclair	Assistant	15
S4612	B004	Seattle	Dave Sinclair	Assistant	10

- Is this table in 2NF? **Explain** your answer.
- Describe and illustrate the process of normalising the data shown in this table to third normal form (3NF).

- First, identify functional dependencies

Fd1: staffNo -> name

Fd2: branchNo -> branchCity

Fd3: staffNo, branchNo -> position, hoursPerWeek

- Then, identify primary key Pk: staffNo, branchNo
- Partial dependencies on primary key exist?

Fd1 and fd2 are partial dependencies on primary key So this relation is not in 2NF

- Remove partial dependencies

The decomposed relations are:

Staff(staffNo , name)

Branch(branchNo , branchCity)

Work(staffNo , branchNo , position, hoursPerWeek)

Since there is no transitive dependencies on primary key, so the above relations are in 3NF

BCNF 算法

- Relation Z(A, B, C, D, E) has functional dependencies:

fd1: A, B $\rightarrow$ C, D, E

fd2: B, C $\rightarrow$ D

Decompose Z to relations in BCNF.

Candidate key for Z is (A, B).

Fd2 violates BCNF definition so it is not in BCNF.

Decompose Z to Z1(B, C, D) Z2(A, B, C, E)

BCNF problem

Student	courseLab	time
111	Database	9:00
112	Database	9:00
113	Database	11:00
111	Multimedia	13:00
113	Multimedia	15:00

- FDs:

Student, courseLab $\rightarrow$ time

time $\rightarrow$ courseLab

- Candidate keys: (Student, courseLab) and (Student, time)

- To change StudentLabTime to BCNF:

Student	time	time	courseLab
111	9:00	9:00	Database
112	9:00	11:00	Database
113	11:00	13:00	Multimedia
111	13:00	15:00	Multimedia
113	15:00		
111	11:00		

However the decomposition is not acceptable because it allows us to record multiple times of the same courseLab against the same student. That is, we have lost the FD: Student, courseLab $\rightarrow$ Time

BCNF problem

- A set of functional dependencies $\{AB \rightarrow C, C \rightarrow B\}$ cannot be represented by a BCNF schema
- A design that eliminates all of these anomalies (but does not conform to BCNF) is possible. This design introduces a new normal form, know as Elementary Key Normal Form (EKNF).

4NF - Example

BranchStaffOwner

branchNo	sName	oName
B003	Ann Beech	Carol Farrel
B003	David Ford	Carol Farrel
B003	Ann Beech	Tina Murphy
B003	David Ford	Tina Murphy

Two nontrivial dependencies:

branchNo →> sName

branchNo →> oName

Not in 4NF

Decompose to 4NF

BranchStaff

branchNo	sName
B003	Ann Beech
B003	David Ford

BranchOwner

branchNo	oName
B003	Carol Farrel
B003	Tina Murphy

29

Exercise 4

Student

studentID	sport	subject
45	Football	English
45	Football	Maths
45	Tennis	English
45	Tennis	Maths
46	Football	English
46	Football	Maths

- Is student relation in 4NF?

There are nontrivial MVDs:

studentID →> sport

studentID →> subject

studentID is not candidate key. So the relation is not in 4NF.

Transaction

Action, or series of actions, carried out by user or application, which reads or updates contents of database.

Properties of Transactions

Four basic (ACID) properties of a transaction are:

- Atomicity**
- Consistency**
- Isolation**
- Durability**

'All or nothing' property
Must transform database from one valid state to another.
Partial effects of incomplete transactions should not be visible to other transactions.
Effects of a committed transaction are permanent and must not be lost because of later failure.

ie, all actions in the transaction are completed or non is completed

ie, transactions execute independently of one another

Transaction example

- Transfer £50 from account A to account B

Read(A)
A = A - 50
Write(A)
Read(B)
B = B + 50
Write(B)

- Atomicity** - shouldn't take money from A without giving it to B
- Consistency** - money isn't lost or gained
- Isolation** - other queries shouldn't see A or B change until completion
- Durability** - the money does not go back to A

Concurrency

- Large databases are used by many people
 - Many transactions to be run on the database
 - It is desirable to let them run at the same time as each other
 - Need to preserve isolation
- If we don't allow for concurrency then transactions are run sequentially
 - Have a queue of transactions
 - Long transactions (e.g. backups) will make others wait for long periods

什么是 Schedule

什么是 serial schedule

- A schedule is a sequence of the operations by a set of concurrent transactions that preserves the order of the operations in each of the individual transactions.
- A serial schedule is a Schedule where operations of each transaction are executed consecutively without any interleaved operations from other transactions.

Serialisability

- Nonserial Schedule: Schedule where operations from set of concurrent transactions are interleaved. i.e., they overlap, transactions might conflict!
- Objective of serialisability is to find nonserial schedules that allow transactions to execute concurrently without interfering with one another.
- In other words, the objective of serialisability is to find nonserial schedules that are equivalent to some serial schedule. Such a schedule is called serialisable.

因为 violates ACID 中的 isolation 所以 concurrency 会出现三种问题

Lost Update problem

后面的更新把前面的更新顶掉了（两个写）

Uncommitted Dependency Problem

回滚的钱已经改变了并被另一个 transaction 读取到了

Inconsistent Analysis Problem

在计算总数时读取了未完成的交易中的变量（一个写一个读）

Two-Phase Locking (2PL): All locking operations precede unlock operation in the transaction

- Every transaction must lock an item (read or write) before accessing it
 - Once a lock has been released, no new items can be locked
1. Growing phase - acquires all locks but cannot release any locks.
 2. Shrinking phase - releases locks but cannot acquire any new locks.

Lost Update Problem

Time	T1	T2	X
t ₁	begin_transaction	begin_transaction	100
t ₂	read(X)	read(X)	100
t ₃	read(X)	X = X+100	100
t ₄	X = X-10	write(X)	200
t ₅	write(X)	commit	90
t ₆	commit		90

Time	T1	T2	X
t ₁		begin_transaction	100
t ₂	begin_transaction	write_lock(X)	100
t ₃	write_lock(X)	read(X)	100
t ₄	WAIT	X:=X+100	100
t ₅	WAIT	write(X)	200
t ₆	WAIT	commit/unlock(X)	200
t ₇	read(X)		200
t ₈	X:=X-10		200
t ₉	write(X)		190
t ₁₀	commit/unlock(X)		190

Time	T3	T4	X
t ₁		begin_transaction	100
t ₃		write_lock(X)	100
t ₄		read(X)	100
t ₅	begin_transaction	X:=X+100	100
t ₄	write_lock(X)	write(X)	200
t ₅	WAIT	rollback/unlock(X)	100
t ₄	read(X)		100
t ₆	X:=X-10		100
t ₇	write(X)		90
t ₈	commit/unlock(X)		90

Time	T5	T6	X	Y	Z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	Sum = 0	100	50	25	0
t ₃	write_lock(X)		100	50	25	0
t ₄	read(X)	read_lock(X)	100	50	25	0
t ₅	X = X-10	WAIT	100	50	25	0
t ₆	write(X)	WAIT	90	50	25	0
t ₇	write_lock(Z)	WAIT	90	50	25	0
t ₈	read(Z)	WAIT	90	50	25	0
t ₉	Z = Z + 10	WAIT	90	50	25	0
t ₁₀	write(Z)	WAIT	90	50	35	0
t ₁₁	Commit/unlock(X,Z)	WAIT	90	50	35	0
t ₁₂		read(X)	90	50	35	0
t ₁₃		sum = sum + X	90	50	35	90
t ₁₄		read_lock(Y)	90	50	35	90
t ₁₅		read(Y)	90	50	35	90
t ₁₆		sum = sum + Y	90	50	35	140
t ₁₇		read_lock(Z)	90	50	35	140
t ₁₈		read(Z)	90	50	35	140
t ₁₉		sum = sum + Z	90	50	35	175
t ₂₀		Commit/unlock(X,Y,Z)	90	50	35	175

• Cascading rollback

a transaction (T1) causes a failure and a rollback must be performed. Other transactions dependent on T1's actions must also be rolled back, thus causing a cascading effect.

One transaction's failure causes many to fail

如何解决 cascading rollback

Postpone the release of all locks until end of the transaction. (Rigorous 2PL)

• Deadlocks

A deadlock occurs when two (or more) transactions are each waiting for locks held by the other to be released.

解决 deadlock

• Timeouts

• Deadlock prevention

Database recovery

Logs

• Contains information about all updates to database:

- Transaction records.
- Checkpoint records.

• Often used for other purposes (for example, auditing)

Log File

• Transaction records contain:

- Transaction identifier.
- Type of log record, (transaction start, insert, update, delete, abort, commit).
- Identifier of data item affected by database action (insert, delete, and update operations).
- Before-image of data item.
- After-image of data item.
- Log management information.

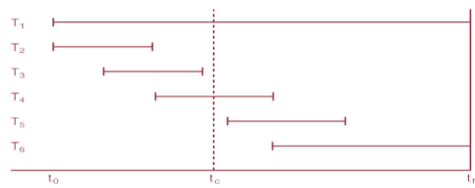
• Checkpoint

Point of synchronization between database and log file. All buffers are force-written to secondary storage.

• Checkpoint record is created containing identifiers of all active transactions.

• When failure occurs, redo all transactions that committed since the checkpoint and undo all transactions active at time of crash

Example



- checkpoint at time t_c , changes made by T_2 and T_3 have been written to secondary storage.
- Thus:
 - only redo T_4 and T_5 .
 - undo transactions T_1 and T_6 .

Distributed DBMSs

Personal databases

目的: allow the user to manage (store, update, delete, and retrieve) small amounts of data in an efficient manner.

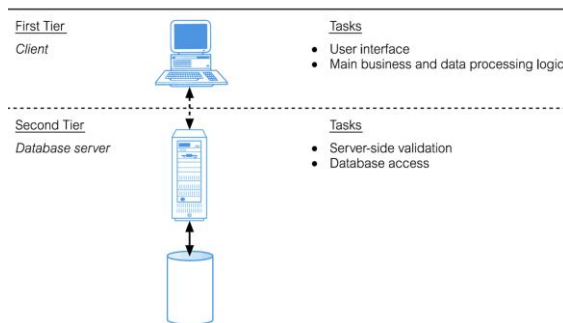
利弊: 能够增加个人使用效率但是不易于分享

Two-Tier Client/Server Architecture

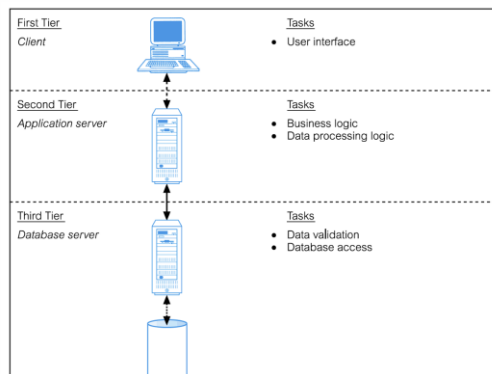
- Computers (client) connected over wired or wireless local area network (LAN)

- The database itself and the DBMS are stored on a central device called the database server, which is also connected to the network.

Two-Tier Client/Server Architecture



Three-Tier Client-Server Architecture



Distributed Database

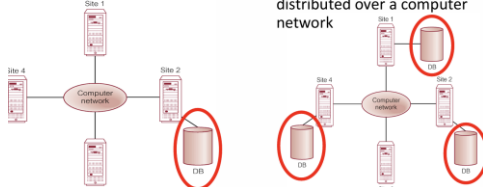
A logically interrelated collection of shared data (and a description of this data), physically spread over a computer network.

Distributed DBMS

Software system that permits the management of the distributed database and makes the distribution transparent to users.

Where is the difference?

- Distributed Processing:
 - centralized database
 - that can be accessed over a computer network
- Distributed DBMS:
 - Single logical database that is split into fragments
 - These fragments are distributed over a computer network



两种数据库的种类

Homogeneous DDBMS (同构分布式数据库系统)

- All sites use same DBMS product.
- Much easier to design and manage.
- Approach provides incremental growth and allows increased performance.

Heterogeneous DDBMS (异构分布式数据库系统)

- Sites may run different DBMS products, with possibly different underlying data models.
- Occurs when sites have implemented their own databases and integration is considered later.
 - Translations are required to allow for:
 - Different hardware.
 - Different DBMS products.
 - Different hardware and different DBMS products.
- Solution??
 - Gateways: a piece of software that causes one DBMS to look like another.

Functions of a DDBMS

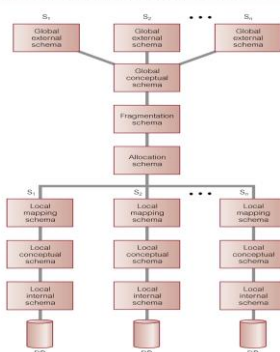
- Expect DDBMS to have at least the functionality of a DBMS.
- Also to have following functionality:
 - Extended communication services.
 - Extended Data Dictionary.
 - Distributed query processing.
 - Extended concurrency control.
 - Extended recovery services.

Reference Architecture for DDBMS

A reference architecture consists of:

- Set of global external schemas.
- Global conceptual schema (GCS).
- Fragmentation schema and allocation schema.
- Set of schemas for each local DBMS conforming to 3 level ANSI/SPARC.

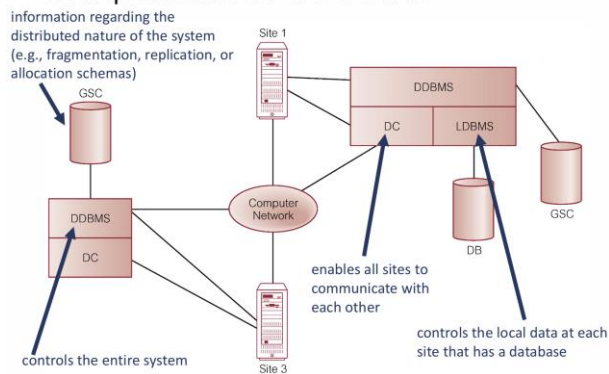
Reference Architecture for DDBMS



Component Architecture for a DDBMS(个人认为比上面那个结构重点)

1. Local DBMS (LDBMS): standard DBMS, responsible for controlling the local data at each site that has a database
2. Data Communications (DC): software enabling all sites to communicate with each other.
3. Global System Catalogue (GSC): holds information specific to the distributed nature of the system, such as the fragmentation, replication, and allocation schemas (it can be managed as a distributed database).
4. Distributed DBMS (DDBMS): the controlling unit of the entire system.

Components of a DDBMS



Distributed Database Design – key issues

- Fragmentation

Relation may be divided into a number of sub-relations, which are then distributed (a fragment is a logical data unit).

- Allocation

Each fragment is stored at the site with “optimal” distribution.

- Replication

Copy of fragment may be maintained at several sites

Disadvantages of Fragmentation

- Performance: the performance of global applications that require data from several fragments located at different sites may be slower.
- Integrity: Integrity control may be more difficult if data and functional dependencies are fragmented and located at different sites.

Fragmentation's correctness

1. Compose all of the fragments should get a complete relation 必须确保能保存所有数据
2. Reconstruction : it must be possible to define a relational operation that will reconstruct the complete relation from fragments. This rule ensures that functional dependencies are preserved. 必须能够还原原关系
3. Disjointness: 各处的数据不重复

Types of Fragmentation

- Horizontal



- Vertical



- Mixed



这里结合 relation algebra 理解

数据分配方式 (Data Allocation Strategies)

Centralized

Fragmented (or Partitioned)

Complete Replication

Selective Replication

Hiding DDBMS complexity, is called transparency

Main types:

1. Distribution Transparency

Distribution transparency allows the user to perceive the database as a single, logical entity.

If distribution transparency is available, then the user does not need to know:

- how data is fragmented (fragmentation transparency),
- or the location of data items (location transparency)

2. Transaction Transparency

Ensures that all distributed transactions maintain distributed database's integrity and consistency.

3. Performance Transparency

- DDBMS must perform as if it were a centralized DBMS.
- DDBMS should not suffer any performance degradation due to distributed architecture.
- DDBMS should determine most cost-effective strategy to execute a request
- Distributed Query Processor (DQP)
 - maps data request into ordered sequence of operations on local databases.
- DQP has to decide: • which fragment to access; • which copy of a fragment to use; • which location to use.

DQP 如何决定查询方式的

Performance Transparency - Example

Property(propNo, city) 10000 records in London
Client(clientNo, maxPrice) 100000 records in Glasgow
Viewing(propNo, clientNo) 1000000 records in London

List the properties in Aberdeen that have been viewed by clients who have a maximum price limit greater than £200,000:

```
SELECT p.propNo  
FROM Property p, Client c, Viewing v  
WHERE c.clientNo = v.clientNo AND  
p.propNo = v.propNo AND  
p.city='Aberdeen' AND c.maxPrice > 200000;
```

- **Strategy 3:** Join the Property and Viewing relations at London, select tuples for Aberdeen properties and then, for each of these tuples in turn, check at Glasgow to determine if the associated client's maxPrice > £200,000. The check for each tuple involves two messages: a query and a response.
Time = $100\,000 * (1 + 100/10\,000) + 100\,000 * 1 \cong 2.3$ days

Performance Transparency - Example

Assume:

- Each tuple in each relation is 100 characters long.
- 10 renters with maximum price greater than £200,000.
- 100 000 viewings for properties in Aberdeen.
- Computation time negligible compared to communication time.

- Data transmission rate is 10,000 characters per second and 1 second access delay to send a message from one site to another.

- **Strategy 4:** Select clients with maxPrice > £200,000 at Glasgow and, for each one found, check at London to see if there is a viewing involving that client and an Aberdeen property. Again, two messages are needed:
Time = $10 * (1 + 100/10\,000) + 10 * 1 \cong 20$ seconds

- **Strategy 1:** Move the Client relation to London and process query there:

Time = $1 + (100\,000 * 100/10\,000) \cong 16.7$ minutes

- **Strategy 2:** Move the Property and Viewing relations to Glasgow and process query there:

Time = $2 + [(1\,000\,000 + 10\,000) * 100/10\,000] \cong 28$ hours

- **Strategy 5:** Join Property and Viewing relations at London, select Aberdeen properties, project result over propertyNo and clientNo, and move this result to Glasgow for matching with maxPrice > £200,000. For simplicity, we assume that the projected result is still 100 characters long:
Time = $1 + (100\,000 * 100/10\,000) \cong 16.7$ minutes

- **Strategy 6:** Select clients with maxPrice > £200,000 at Glasgow and move the result to London for matching with Aberdeen properties:
Time = $1 + (10 * 100/10\,000) \cong 1$ second

Communication Time = $C_0 + (\text{no_of_bits_in_message}/\text{transmission_rate})$

DDBMSs 的优缺点

良好的结构

可分享性和本地扩展性

可访问性, 可靠性

更好的查询

方便扩展

但

复杂，数据规模大的时候贵

安全问题，完整性约束更加困难，标准不统一，数据库设计更加困难

数据库安全+数据伦理

Database security

六个数据库安全措施

1.authorization

2.access control

3.views (视图)

The view provides a security mechanism by hiding parts of the database from certain users. A view can be defined over several relations with a user being granted the appropriate privilege to use it, but not to use the base relations.

4.backup and recovery

5.integrity

Prevents data from becoming invalid, and hence giving misleading or incorrect results.

6.encryption (加密)

the encoding of the data by a special algorithm that renders the data unreadable by any program without the decryption key

sql 注入

1.通过注释

2.通过加恒成立条件

3.通过加 union 条件

4.通过盲注，观察服务器反映判断正确或错误，然后用字符串一个一个猜管理员的密码，无法直接获取的情况下使用

预防

- Using prepared statements (parameterized statements)
- Escaping all user supplied input
- Pattern check
- Database permissions
- Detecting SQL injection vulnerabilities, manually or using web vulnerability scanner.

数据伦理的 4 个 principle: ownership, transparency, data privacy, accountability

可扩展标记语言 XML

eXtensible Markup Language (XML)

XML – Elements

XML - Elements

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<Bookstore>
  <Book ISBN="0590353403" Price="20" Edition="1st">
    <Title>Harry Potter and the Philosopher's Stone</Title>
    <Author>
      <First_Name>J.K.</First_Name>
      <Last_Name>Rowling</Last_Name>
    </Author>
  </Book>
</Bookstore>
```

- Elements, or tags, are most common form of markup.
- First element must be a root element, which can contain other (sub)elements.
- XML document must have one root element **<Bookstore>**.
 - start-tag **<Bookstore>**
 - end-tag **</Bookstore>**
- XML elements are case sensitive.
- An element can be empty, in which case it can be abbreviated to **<EMPTYELEMENT/>**
- Elements must be properly nested.

XML - Attributes

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<Bookstore>
  <Book ISBN="0590353403" Price="20" Edition="1st">
    <Title>Harry Potter and the Philosopher's Stone</Title>
    <Author>
      <First_Name>J.K.</First_Name>
      <Last_Name>Rowling</Last_Name>
    </Author>
  </Book>
</Bookstore>
```

- Attributes are name-value pairs that contain descriptive information about an element.
- Attribute is placed inside start-tag after corresponding element name with the attribute value enclosed in double quotes.
<Book ISBN="0590353403" Price="20" Edition="1st">
- A given attribute may only occur once within a tag, while sub-elements with same tag may be repeated.

XML – Other Sections

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<Bookstore>
  <Book ISBN="0590353403" Price="20" Edition="1st">
    <Title>Harry Potter and the Philosopher's Stone</Title>
    <Author>
      <First_Name>J.K.</First_Name>
      <Last_Name>Rowling</Last_Name>
    </Author>
  </Book>
</Bookstore>
```

- **XML declaration:** optional at start of XML document.
E.g., **<?xml version="1.0" encoding="UTF-8" standalone="no" ?>**
- **Entity references:** serve various purposes, such as shortcuts to often repeated text or to distinguish reserved characters from content.
E.g., **<** is the same as **<**
- **Comments:** enclosed in **<!--** and **-->** tags.
E.g., **<!--this is a comment-->**

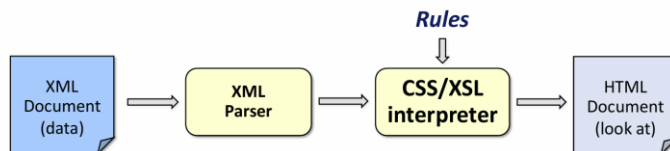
- **XML – Ordering**
 - elements are ordered.
 - attributes are unordered.

Displaying XML

Displaying XML

Use rule-based languages to translate to HTML

- **Cascading StyleSheets (CSS):** alternative rendering to the tags
- **eXtensible Stylesheet Language (XSL):** defines how data is rendered



DTD (document type definitions)

Defines the valid syntax of an XML document

一、DTDs: 元素类型声明 (Element Type Declarations)

用于指定哪些元素可以出现在 XML 文档中, 以及它们出现的次数规则和排列顺序。

符号	含义	示例说明
*	0 次或多次	<Book*> 表示 Book 可以出现 0 次、1 次或多次
+	1 次或多次	<Book+> 表示至少要有一个 Book
?	0 次或 1 次	<Title?> 表示 Title 可有可无, 但只能出现一次
无符号	必须出现一次	<Title> 表示 Title 元素必须出现一次

✅ 顺序与排列:

- 元素名之间使用 **逗号 (,)** 分隔 ⇒ 表示它们必须**按顺序出现**
例: **<!ELEMENT Book (Title, Price)>** ⇒ 必须先写 Title, 再写 Price
- 如果没有逗号, 表示这些元素可以**以任意顺序出现**

✅ #PCDATA 的含义:

- #PCDATA: **可解析字符数据**, 代表元素内容是纯文本 (可以被 XML 解析器处理的文字内容)

二、DTDs: 属性列表声明 (Attribute List Declarations)

作用:

规定某个元素可以拥有哪些属性、属性的数据类型、是否必需、默认值等。

✅ 常见类型说明:

类型 含义

CDATA 普通字符数据 (任何文本都可以)

ID 唯一标识符 (在文档中必须唯一)

IDREF 引用其他元素的 ID

IDREFS 引用多个 ID (空格分隔)

名称列表 属性的值必须是某个特定选项, 例如: `(male`

三、DTDs: 元素标识与引用 (ID, IDREF, IDREFS)

作用:

用来在 XML 文档中标识某个元素, 并允许其他元素引用它, 实现类似“主键/外键”的逻辑关联。

✅ 类型说明:

类型 用途

ID 唯一标识一个元素, 类似于数据库的主键 (Primary Key)

IDREF 引用某个 ID, 表示一对一关系

IDREFS 引用多个 ID, 表示一对多关系 (以空格分隔)

✅ 示例: 模拟“一个机构有多个员工”的关系

```
<!ATTLIST STAFF staffNo ID #REQUIRED>
```

```
<!ATTLIST BRANCH staff IDREFS #IMPLIED>
```

解释:

- STAFF 元素有一个唯一的 staffNo 属性 (ID 类型) ⇒ 每个员工有唯一编号
- BRANCH 元素有一个 staff 属性 (IDREFS 类型) ⇒ 它可以引用多个 STAFF 的 staffNo ⇒ 即一个机构对应多个员工

✅ 举个完整的例子:

```
<STAFF staffNo="S001"/>
```

```
<STAFF staffNo="S002"/>
```

```
<BRANCH staff="S001 S002"/>
```

这里 BRANCH 引用了两个 STAFF 的 ID, 建立了它们之间的关系。

XML Schema Definition (XSD)

✅ DTD 的局限性 (为什么不推荐 DTD):

1. **不是 XML 语法本身写的:**
 - DTD 使用类 BNF 的语法, 不符合 XML 格式, 难以统一处理。
2. **不支持命名空间 (namespaces):**
 - 在多个 XML 文档整合或协作时, 很难处理名称冲突。

3. 数据类型支持非常有限:

- DTD 只能区分字符数据 (#PCDATA) 或 ID/IDREF, 没有类型限制, 比如不能指定整数、布尔型等。

4. 过时的技术 (Old):

- 随着 Web 数据交换的复杂度增加, DTD 显得功能薄弱

特性	DTD	XSD
编写语法	非 XML (EBNF 语法)	XML 本身
数据类型支持	非常有限 (主要是字符串)	非常丰富 (整数、浮点、布尔等)
命名空间支持	不支持	支持
可读性与工具兼容性	差	高, 可用 XML 工具处理
是否推荐使用	已过时, 不推荐	推荐使用

1. XSD 数据类型定义 (XSD Types)

```
<xsd:attribute name="Price" type="xsd:integer" use="required"/>
```

```
<Book ISBN="0590353403" Price="100"/>
```

这个定义说明:

- 在 XML 中必须包含 Price 属性
- 它的值必须是 **整数类型 (xsd:integer)**
- 如果填写非整数, 比如 Price="foo" ⇒ 会验证失败

2. XSD 主键定义 <xsd:key>

```
<xsd:key name="BookKey">  
  <xsd:selector xpath="Book"/>  
  <xsd:field xpath="@ISBN"/>  
</xsd:key>
```

```
<xsd:key name="AuthorKey">  
  <xsd:selector xpath="Author"/>  
  <xsd:field xpath="@Ident"/>  
</xsd:key>
```

```
<Author Ident="HG">  
  <First_Name>Hector</First_Name>  
</Author>
```

```
<Book ISBN="9781292061184" Price="35">  
  <Title>Database Systems</Title>  
</Book>
```

3. XSD 外键定义 <xsd:keyref>

```
<xsd:keyref name="AuthorKeyRef" refer="AuthorKey">  
  <xsd:selector xpath="Book/Authors/Auth"/>  
  <xsd:field xpath="@authIdent"/>
```

</xsd:keyref>

<Author Ident="TC">...</Author>

<Author Ident="CB">...</Author>

<Book ISBN="9781292061184" Price="35">

...

<Authors>

<Auth authIdent="TC"/>

<Auth authIdent="CB"/>

</Authors>

</Book>

这个定义说明:

- 在 XML 中的 <Auth> 元素的 authIdent 属性必须引用一个合法的 Author 的 Ident
- AuthorKeyRef 是对 AuthorKey 的引用

XSD 简单类型 (simpleType)

<xsd:element name="STAFFNO" type="xsd:string"/>

<xsd:element name="DOB" type="xsd:date"/>

<xsd:element name="SALARY" type="xsd:decimal"/>

<xsd:attribute name="branchNo" type="xsd:string"/>

属性必须写在元素的后面或内部的最后

XSD 复杂类型 (complexType)

<xsd:element name="Book">

<xsd:complexType>

<xsd:sequence>

<xsd:element name="Title" type="xsd:string"/>

</xsd:sequence>

<xsd:attribute name="Price" type="xsd:integer" use="required"/>

</xsd:complexType>

</xsd:element>

<Book Price="100">

<Title>Data Systems</Title>

</Book>

元素的出现次数约束 (Cardinality: minOccurs, maxOccurs)

约束形式

含义

minOccurs="0"

可以不出现 (即“可选”)

maxOccurs="1"

最多出现一次 (默认值)

maxOccurs="unbounded"

可以出现任意多次

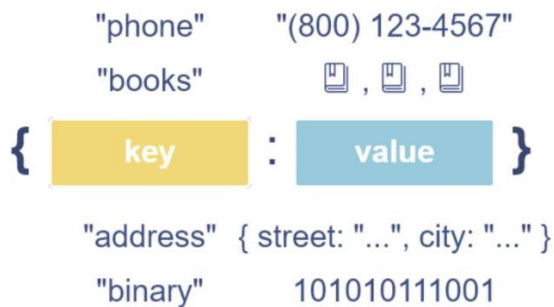
方面表现出色。然而，如果你的应用需要严格的数据一致性，或者需要支持复杂的事务，那么传统的关系数据库可能是更好的选择。

- **Basically Available:**
 - Any request => An answer
 - Even in a changing state
- **Soft State:**
 - Opposite to Durability.
 - System's state (servers or data) could change over time (without any update)
- **Eventually consistent:**
 - With time, data can be consistent
 - Updates have to be propagated

优缺点

- Key features:
 - Flexible schema
 - Quicker/cheaper to set up
 - Massive scalability
 - Relaxed consistency --> higher performance & availability
- Disadvantages:
 - No declarative query language --> more programming
 - Relaxed consistency --> fewer guarantees

Key-value stores



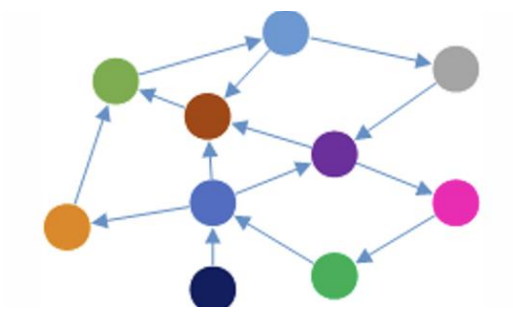
Document stores

- Data model: (key, document) pairs
- Helpful in managing semi-structured data, and data are typically stored in JSON, XML,

or other semi-structured formats

Graph database systems

- Data model: nodes and edges
- Nodes may have properties (including ID)



What NoSQL should NOT be used for

- Anything that requires frequent updates as well as reads, or that requires high integrity and atomicity (ACID properties) 频繁更新+强完整性要求
- Examples are similar to transaction databases for inventory and financial records 涉及交易操作
- This is not just a question of massive data or distributed processing! 不只是数据量大就用 nosql 而是是否需要强数据保障
- There are large, distributed relational databases like Visa or Amazon that need more structured data with transaction semantics 因为它们的数据处理需要严格的交易语义。
- These applications are better suited to relational databases even at large scale 即使在大规模下，金融、库存、订单系统更适合使用关系型数据库。